



FGE

Rapport :

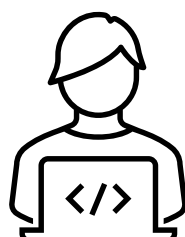
Founzo

2025



Table des matières

I.	Introduction	2
A.	Motivations	2
B.	Présentation	2
II.	Architecture	3
A.	Présentation de la pipeline	3
III.	Allocations mémoires massives	3
A.	Choix de l'allocateur	4
B.	Implémentation.....	5
C.	Représentation sur GPU	5
D.	Stratégie de transfert	5
IV.	Références	5
V.	Conclusion.....	5



I. Introduction

A. Motivations

L'objectif de ce rapport est principalement un entraînement pour améliorer mes qualités en écriture ainsi qu'en transfert de connaissances. Il est important de pouvoir transmettre un savoir de manière efficace tout en assurant une lecture agréable. Cela repose sur un texte structuré et clair ainsi que sur des schémas véhiculant une ou plusieurs informations précises.

La rédaction d'un rapport permet de faire le point sur des connaissances acquises ainsi que sur des idées explorées. Ces différents éléments peuvent alors être rassemblés pour avoir une vue précise et globale sur l'état actuel. De plus, c'est l'occasion de pouvoir réfléchir à de nouvelles perspectives nées par des motivations réfléchies. Ces perspectives peuvent porter sur des fonctionnalités ou bien des améliorations de performances. Enfin, cela permet de mieux guider nos prochains choix d'implémentation afin de réduire la nécessité de rétro pédaler.

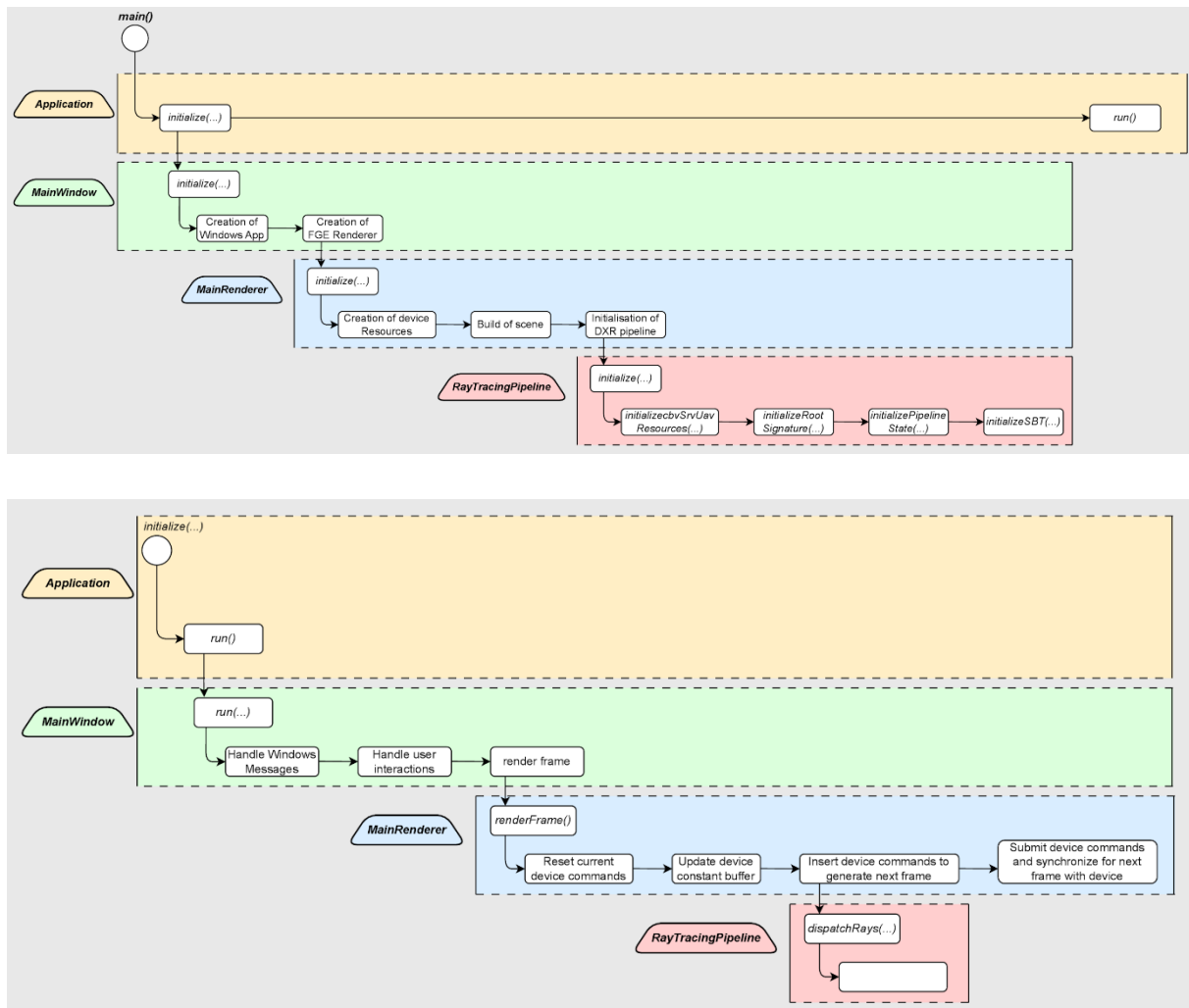
Par ailleurs, on peut y mesurer les progrès atteints et prendre conscience des défauts à corriger qu'ils soient méthodologiques ou fonctionnels. Par conséquent, la rédaction d'un rapport est à faire pendant la durée du projet et bien sûr à la fin.

Ce document sert par la même occasion de documentation, toute personne peut lire ce rapport et comprendre le fonctionnement du projet tout en ayant à l'esprit la raison des différents choix réalisés. De plus, les développeurs peuvent également le consulter pour se rafraîchir l'esprit après une longue période d'interruption du projet.

B. Présentation

II. Architecture

A. Présentation de la pipeline



III. Allocations mémoires massives

Dans le cadre d'une scène dynamique, il est nécessaire de pouvoir insérer, supprimer ou modifier un objet. Chaque objet peut être petit ou gros, principalement due à la représentation du mesh. De plus, une scène peut avoir des centaines ou des milliers d'objets où chacun peut nécessiter une mise à jour. Il s'agit d'un véritable défi puisqu'il faut pouvoir avoir la capacité d'allouer et libérer de la mémoire à destination de ces objets. De plus, il est nécessaire de pouvoir déplacer et mettre à jour la représentation mémoire host sur le device où celui-ci possède des contraintes d'alignements mémoires spécifiques. Face à ce défi, on se voit dans la contrainte de réaliser un allocateur personnalisé permettant au minima un « malloc/free » pour un objet. De plus, une fois transférer sur le device, le GPU doit pouvoir accéder aux composantes des objets de manière performante.

A. Choix de l'allocateur

Pour mettre en place un tel système, nous avons réalisé une petite étude de différentes stratégies qui nous guidera dans notre choix d'implémentation. A noter, que chacune de ces méthodes possèdent des alternatives mais nous nous intéressons à l'idée générale.

1. Suballocation + freelist

Cette première méthode repose sur un méga buffer alloué que l'on va ensuite sous allouer par notre allocateur. Pour cela, une liste contient une représentation des allocations possibles. Chaque élément représente l'offset à partir duquel le buffer est libre puis la taille permettant de savoir jusqu'à où il se termine. Le schéma suivant met en avant la représentation de cette structure.

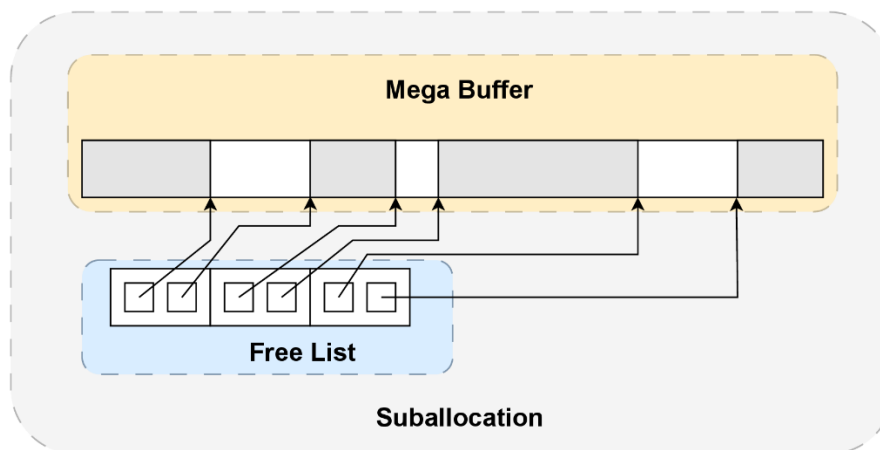


Figure 1: Représentation d'une structure d'allocation de type Suballocation. L'allocateur peut allouer un total de $\text{sizeof}(\text{Mega Buffer})$ octets.

L'allocation se fait en parcourant les éléments de la Free List pour sélectionner une plage capable d'accueillir l'allocations. Plusieurs stratégies peuvent être utilisé pour sélectionner la plage. La sélection comme présenté dans cette représentation est en $O(n)$. La libération d'un élément est ici en $O(1)$. Après ces différentes opérations il est évidemment nécessaire de mettre à jour la Free List. Le principale désavantage de cette structure est la fragmentation qui peut impliquer des trous que l'on n'utilise pas. En revanche, cette structure est efficace pour stocker des objets de tailles largement variées.

2. Page-based allocation

Cette stratégie repose également sur un méga buffer, à la différence qu'il est déjà pré-découpé en pages de tailles fixes. Cela permet de faciliter la sélection puisque l'on récupère le premier élément de Free pool qui stocke les indices des pages non utilisées. Par conséquent, l'allocation se fait en $O(1)$ tout comme la libération. On peut également facilement visualiser l'occupation des pages via une bitmap où chaque bit indique son utilisation ou non. C'est particulièrement utile dans le cas d'analyse de performances et de débogage. Le schéma ci-dessous représente cette structure.

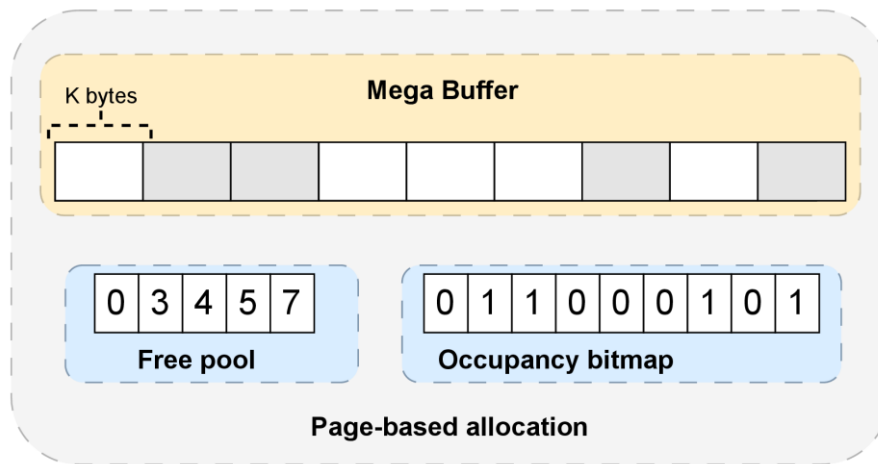


Figure 2: Représentation d'une structure d'allocation de type Page-based. L'allocateur peut allouer un total de $\text{sizeof}(\text{Mega Buffer})/K$ éléments de tailles unitaire de K octets.

Toutefois, cette structure présente un défaut majeur. Dans le cas où on alloue pour une taille supérieure à la taille d'une page, il faut alors allouer plus de 1 page pour un objet. Cela complique alors l'indexation quand on souhaite consulter un élément. De plus, pour des allocations d'une taille largement inférieure à la taille d'une page, il y a alors un gâchis mémoire importante. Pour réduire cet impact, on peut faire plusieurs allocateurs de ce type mais avec une taille de pages différentes. Puis, l'allocation serait affectée à l'allocateur ayant la taille de pages la plus proche de ce qui est demandé.

3. Double-buffering

4. Décision prise

B. Implémentation

C. Représentation sur GPU

D. Stratégie de transfert

IV. Références

[1] Moscow Aviation Institute. 2000-2012. GLPK (GNU Linear Programming Kit). Lien : <https://www.gnu.org/software/glpk/>

[2] Google. 2025. OR-Tools - Google Optimization Tools. Lien : <https://developers.google.com/optimization>

V. Conclusion

Founzo.

Etudiant en M1 CHPS et en CMI à l'Université de Reims Champagne Ardenne.

Alternant au laboratoire LIICIS de Reims, « *Apprenti Chercheur* ».